The `association=` method assigns an associated object to this object. Behind the scenes, this means extracting the primary key from this object and setting the associate object's foreign key to the same value.

```
@supplier.account = @account
```

4.2.1.3 `build_association(attributes = {})`

The `build_association` method returns a new object of the associated type. This object will be instantiated from the passed attributes, and the link through its foreign key will be set, but the associated object will *not* yet be saved.

```
@account = @supplier.build_account(terms: "Net 30")
```

4.2.1.4 `create_association(attributes = {})`

The `create_association` method returns a new object of the associated type. This object will be instantiated from the passed attributes, the link through its foreign key will be set, and, once it passes all of the validations specified on the associated model, the associated object *will* be saved.

```
@account = @supplier.create_account(terms: "Net 30")
```

4.2.1.5 `create_association!(attributes = {})`

Does the same as `create_association` above, but raises `ActiveRecord::RecordInvalid` if the record is invalid.

4.2.2 Options for `has_one`

While Rails uses intelligent defaults that will work well in most situations, there may be times when you want to customize the behavior of the `has_one` association reference. Such customizations can easily be accomplished by passing options when you create the association. For example, this association uses two such options:

```
class Supplier < ActiveRecord::Base
  has_one :account, class_name: "Billing", dependent: :nullify
end
```

The `has_one` association supports these options:

- `:as`
- `:autosave`
- `:class_name`
- `:dependent`
- `:foreign_key`
- `:inverse_of`
- `:primary_key`
- `:source`
- `:source_type`
- `:through`
- `:validate`

4.2.2.1 `:as`

Setting the `:as` option indicates that this is a polymorphic association. Polymorphic associations were discussed in detail earlier in this guide.

4.2.2.2 `:autosave`

If you set the `:autosave` option to `true`, Rails will save any loaded members and destroy members that are marked for destruction whenever you save the parent object.

4.2.2.3 `:class_name`

If the name of the other model cannot be derived from the association name, you can use the `:class_name` option to supply the model name. For example, if a supplier has an account, but the actual name of the model containing accounts is `Billing`, you'd set things up this way:

```
class Supplier < ActiveRecord::Base
  has_one :account, class_name: "Billing"
end
```

4.2.2.4 `:dependent`